

Interference-Aware Routing Schemes

Autonomous Networks Research Group (ANRG)
University of Southern California

1 Overview

When tasks in a DAG are placed on different nodes, data transfers traverse multi-hop wireless paths where concurrent transmissions interfere with each other. The **routing scheme** decides *which path* each transfer takes and—in future work—*when* it begins. All schemes listed here operate on top of HEFT task placement and the `csma.bianchi` 802.11 interference model.

The schemes fall into three families:

Family	Schemes	When routes are decided
Baseline	W, S	Once, at topology load time (cached)
Static greedy	GS, GC, GB, GO	Once, at DAG injection (after HEFT)
Dynamic greedy	GSD	At each <code>transfer_start</code> event
Deferral (<i>planned</i>)	GSD-D, GS-D, ...	At <code>transfer_start</code> , may post-pone

2 Baseline Schemes

2.1 W — Widest-Path

Property	Value
Class	<code>WidestPathRouting</code>
Objective	Maximize bottleneck (min-link) bandwidth
Algorithm	Modified Dijkstra with max-min semantics
Interference awareness	None—ignores concurrent traffic

Finds the path whose weakest link has the highest bandwidth. Good when link capacities vary widely (e.g., short vs. long hops have different PHY rates) but oblivious to runtime contention.

2.2 S — Shortest-Path

Property	Value
Class	<code>ShortestPathRouting</code>
Objective	Minimize total path latency (sum of link latencies)
Algorithm	Standard Dijkstra on link latencies
Interference awareness	None

Minimizes propagation delay. In grids where all links have equal latency this degenerates to minimum hop count. Tends to concentrate traffic on short paths, which can increase contention.

3 Static Greedy Schemes (GS, GC, GB, GO)

All four variants share the same two-stage architecture:

1. **HEFT** decides task-to-node placement and produces timing estimates.
2. **Greedy route planner** (`InterferenceAwareRouting.plan_routes()`) iterates over inter-node flows in a variant-specific order and, for each flow, picks the candidate path that maximizes *total system throughput* across all concurrently-active flows (including previously routed ones).

Routes are computed once at DAG injection and are fixed for the entire execution. The variants differ only in the order flows are considered.

3.1 GS — Greedy by Start Time

Flow ordering	Ascending estimated transfer start time
Rationale	Route earlier flows first; later flows adapt around them
CLI	<code>--routing interference_aware --greedy-order start</code>

Earliest-first is the default and most natural ordering: it mirrors the temporal sequence in which flows will actually appear during execution.

3.2 GC — Greedy by Criticality

Flow ordering	Descending HEFT upward-rank of the <i>destination</i> task
Rationale	Critical-path flows get first pick of low-interference routes
CLI	<code>--routing interference_aware --greedy-order criticality</code>

Upward rank (**ranku**) measures how much downstream work depends on a task. Routing the most critical transfers first gives them the best paths, at the expense of less critical flows.

3.3 GB — Greedy by Bytes

Flow ordering	Descending data size
Rationale	Large transfers dominate makespan; give them the best routes
CLI	<code>--routing interference_aware --greedy-order bytes</code>

When transfer sizes vary, the largest flows contribute most to makespan. GB prioritizes them. Less effective when all transfers are the same size (as in many synthetic benchmarks).

3.4 GO — Greedy by Overlap

Flow ordering	Descending overlap degree (total temporal overlap with other flows)
Rationale	Flows with the most contention should be routed first
CLI	<code>--routing interference_aware --greedy-order overlap</code>

Overlap degree is the sum of time-window overlaps between a flow and all other flows (using HEFT timing estimates). Flows competing for airtime with many others get priority.

3.5 Scoring Function (All Four Variants)

For each candidate path, the planner computes:

$$score = \sum_{concurrent\ flows} effective_bottleneck_bandwidth$$

where effective bandwidth accounts for:

- **Interference factor** from `csmabianchi` (SINR degradation from hidden terminals + Bianchi MAC contention from conflict-graph neighbors)
- **Per-link fair sharing** (N flows on a link each get $1/N$ of the base bandwidth)

The candidate with the highest total-throughput score wins.

4 Dynamic Greedy Scheme

4.1 GSD — Greedy by Start Time, Dynamic

Class	<code>DynamicInterferenceAwareRouting</code>
Objective	Maximize <i>this flow's</i> bottleneck bandwidth given actual state
When computed	At each <code>transfer_start</code> event
State used	Actual set of active links and per-link transfer counts
CLI	<code>--routing interference_aware_dynamic</code>

Unlike the static variants (which estimate concurrency from HEFT timing), GSD sees the *real* network state at the moment a transfer begins. This lets it:

- React to transfers that finished early or late relative to HEFT estimates
- See sibling transfers that started moments earlier at the same sim-time
- Avoid paths that are congested *right now*, not just predicted to be

Trade-off: GSD optimizes for *this individual flow's* bottleneck bandwidth rather than total system throughput. It cannot coordinate across flows the way the static planner can.

Delegation: During HEFT scheduling (before execution), GSD delegates to `WidestPathRouting` for bandwidth estimates, since no runtime state is available yet.

5 Deferral (Planned—Not Yet Implemented)

5.1 Concept

All current schemes start a transfer as soon as its source task completes. **Deferral** adds an option: if starting a transfer *now* would severely degrade an already-active flow (or the new flow itself) due to interference, the engine can **postpone** the new transfer until conditions improve.

5.2 Potential Variants

GSD-D (Dynamic + Deferral): At `transfer_start`, if the best available path scores below a threshold (e.g., $< 50\%$ of the no-contention bandwidth), defer the transfer and re-evaluate after the next `transfer_complete` event.

GS-D (Static + Deferral): Use GS route planning but allow the execution engine to hold back flows whose planned route is currently congested.

5.3 Key Design Questions

1. **Deferral trigger:** Score threshold? Comparison to no-contention baseline? Always defer if any link on the path is above N concurrent flows?
2. **Re-evaluation:** On every `transfer_complete`? After a fixed timeout?
3. **Starvation:** How to guarantee deferred transfers eventually execute (maximum deferral budget, priority escalation)?
4. **Impact on HEFT:** Deferral changes actual timing relative to HEFT estimates. Does this cascade and hurt downstream tasks?

5.4 Expected Benefit

Deferral is most useful when:

- A few high-bandwidth transfers dominate makespan and would complete faster if not forced to share with small concurrent transfers
- The DAG has enough parallelism that deferring one transfer doesn't immediately stall a critical-path task

6 Configuration Reference

6.1 CLI

```
--routing {direct,widest_path,shortest_path,
           interference_aware,interference_aware_dynamic}
--greedy-order {start,criticality,bytes,overlap}
--routing-hop-cutoff N
--interference {none,proximity,csma_clique,csma_bianchi}
```

6.2 YAML

```
config:
  routing: interference_aware
  greedy_order: start
  routing_hop_cutoff: 4
  interference: csma_bianchi
```

6.3 Shared Parameters

Parameter	Default	Effect
hop_cutoff	4	Maximum hops in candidate path enumeration
max_candidates	20	Maximum candidate paths evaluated per flow

7 Summary

Scheme	Route timing	Objective	Flow ordering	Interference input
W	Load time	Max bottleneck BW	—	None
S	Load time	Min total latency	—	None
GS	DAG inject	Max system throughput	Est. start time	HEFT estimates
GC	DAG inject	Max system throughput	HEFT upward rank	HEFT estimates
GB	DAG inject	Max system throughput	Data size (desc)	HEFT estimates
GO	DAG inject	Max system throughput	Overlap degree	HEFT estimates
GSD	Transfer start	Max flow bottleneck BW	—	Actual active links
GSD-D	Transfer start	Max flow bottleneck BW	—	Actual + deferral